Unit 2
Computational Thinking & Algorithms

Decompose · recognise patterns · abstract · then write the steps

DECOMPOSITION | PATTERN RECOGNITION | ABSTRACTION | BUBBLE & SELECTION SORT | LINEAR & BINARY SEARCH

Self-study companion · every algorithm is traced by hand before you meet Python in Unit 3

0. The habit before the language

Computational thinking is a four-move habit: break the mess apart, notice what repeats, hide the irrelevant, and write a language-free sequence of steps. An **algorithm** is that sequence. **Pseudocode** is the same sequence written with structured words such as IF, FOR, and WHILE so a programmer can later type it in Python without reinventing the logic.

WHY IT MATTERS

Every program you will write in Unit 3 is an algorithm wearing syntax. If the algorithm is wrong, prettier code will not save it. Examiners award marks for the steps, the trace, and the reason you chose binary search rather than linear.

ALGORITHM

A step-by-step logical procedure that solves a problem. Conceptual. Independent of any programming language. May be a numbered list.

PSEUDOCODE

Human-readable draft of an algorithm using plain English plus structured keywords. Semi-formal. Not executable. Aimed at programmers about to code.

WHY BOTHER?

- Structure complex work
- Save time and memory
- Reuse the same idea
- Reduce human error
- Blueprint before code

1. Three strategies you apply in this order

1.1 Decomposition

Split a large task into smaller tasks you can finish and test. Learning a language is not one mountain; it is vocabulary, grammar, tenses, sentence practice, and feedback. A school attendance app is login, class list, mark present/absent, save, report. If a piece is still too big, split it again.

1.2 Pattern recognition

Look for repetition and rules. A star triangle in which row i prints i stars is one rule, not ten separate drawings. Once you see the pattern you write one loop instead of copy-paste.

$$\begin{aligned}&\text{row } 1: *\\&\text{row } 2: **\\&\text{row } i: i \text{ stars}\end{aligned}$$

1.3 Abstraction

Keep what changes the result; drop what does not. Making tea: boil water, add tea, steep, pour. Cup colour and kettle brand are noise. In a library search you keep title, author, and availability — not the colour of the spine.

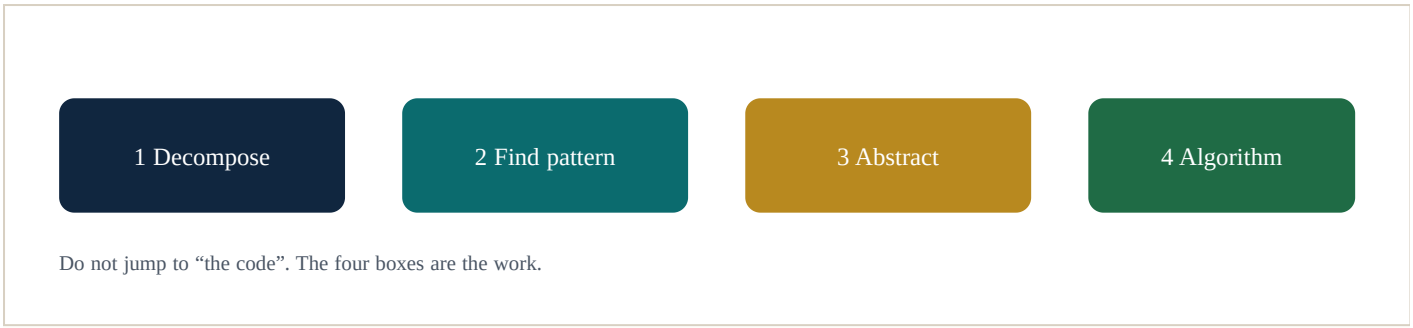


Figure 1. Computational thinking pipeline. The algorithm is the last box, not the first.

2. Sorting — putting a list in order

Sorting arranges data ascending, descending, or by time. Databases, shopping sites, and game leaderboards all sort before they show you anything useful. This unit asks you to master two simple quadratic sorts.

2.1 Bubble sort — neighbours swap until the list is quiet

Walk left to right. Compare each pair of neighbours. If they are in the wrong order, swap. The largest value “bubbles” to the end on pass 1. Repeat, ignoring the already-settled tail.

$$\begin{array}{l}\textbf{for } k \leftarrow 1 \textbf{ to } n-1 \textbf{ do} \\ \quad \textbf{for } ptr \leftarrow 1 \textbf{ to } n-k \textbf{ do} \\ \quad\quad \textbf{if } A[ptr] > A[ptr+1] \textbf{ then swap}\end{array}$$

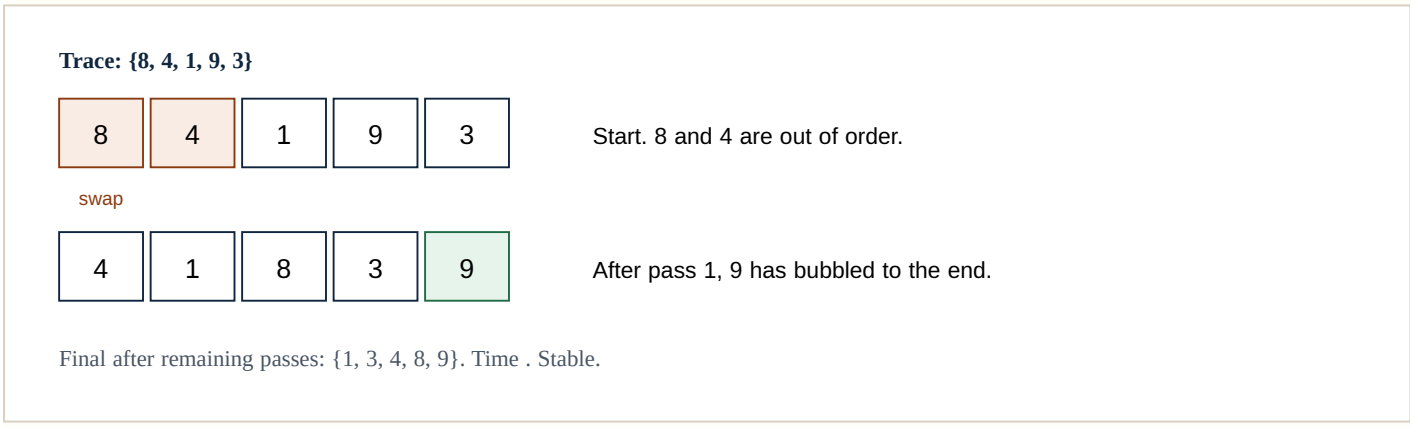


Figure 2. One bubble-sort pass. Shade the settled suffix after each outer loop.

Complexity: about $n(n-1)/2$ comparisons $\Rightarrow O(n^2)$. It is **stable** (equal keys keep their order) and easy to code, but too slow for large n.

2.2 Selection sort — pick the next minimum, plant it

On pass i, scan the unsorted suffix, find the smallest item, and swap it into position i. At most one swap per pass (plus many comparisons).

$$\begin{array}{l}\textbf{for } i \leftarrow 1 \textbf{ to } n-1 \textbf{ do} \\ \quad min \leftarrow i \\ \quad \textbf{for } j \leftarrow i+1 \textbf{ to } n \textbf{ do} \\ \quad\quad \textbf{if } A[j] < A[min] \textbf{ then } min \leftarrow j \\ \quad \textbf{swap } A[i] \textbf{ with } A[min]\end{array}$$

Same $O(n^2)$ family as bubble sort. Usually **not stable**. Use it when swaps are expensive and n is small — you pay for comparisons but almost no movement.

Table 1. Two $O(n^2)$ sorts compared

	Bubble	Selection
Idea	Swap neighbours	Select the next min
Swaps	Many (up to one per compare)	At most one per pass
Stable?	Yes	Usually no
Best use	Teaching, tiny lists, nearly sorted (if you add a “no-swap” flag)	Tiny lists when writing is costly

3. Searching — finding one item

3.1 Linear search — start at the left, walk

Works on **unsorted** data. Compare the key with $A[1]$, then $A[2]$, … until you hit it or fall off the end.

Example: find 7 in $\{8, 3, 6, 1, 7, 2, 4, 5\}$. You inspect five cells and stop. Worst case is n inspections: $O(n)$.

3.2 Binary search — halve a sorted list

The list *must already be sorted*. Keep three markers: beginning, end, and midpoint.

$$MID = \left\lfloor \frac{BEG + END}{2} \right\rfloor$$

If $A[MID]$ is the key, stop. If the key is smaller, throw away the right half ($END \leftarrow MID - 1$). If larger, throw away the left half. Repeat. You need random access (an array), not a one-way linked walk.

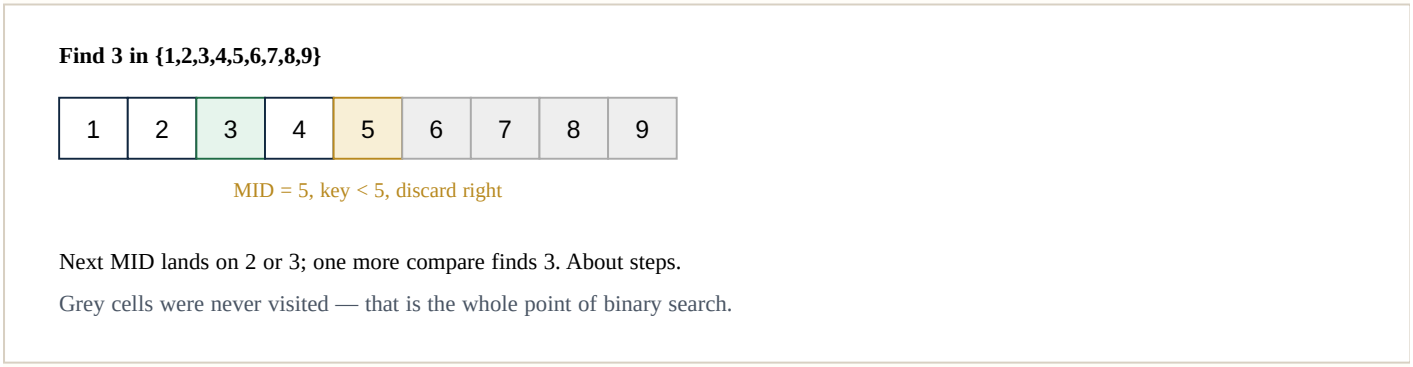


Figure 3. Binary search throws away half the remaining list every time. Complexity $O(\log n)$.

$$T_{\text{linear}}(n) = O(n), \qquad T_{\text{binary}}(n) = O(\log n) \quad \textbf{but binary requires a sorted array.}$$

4. Is the algorithm any good?

CORRECTNESS
It matches the specification, handles edges (empty list, $n = 1$, key missing), and *terminates*. Dry-run with a trace table: columns for line number, each variable, conditions, and output.

EFFICIENCY
Time (how the step-count grows with n) and space (extra memory). Prefer binary over linear on a large sorted catalogue. Prefer $O(n \log n)$ sorts once n leaves the classroom.

CLARITY
Named steps, obvious variables, one job per block. A clever one-liner that nobody can trace is a bad algorithm for this course.

4.1 Mini decision guide

Table 2. Which algorithm do I reach for?

Situation	Choose
Find an item in an unsorted bag	Linear search
Find an item in a sorted array	Binary search
Need the data itself in order	A sort (bubble/selection for tiny n)
Same sub-problem appears again and again	Remember the answer (the textbook flags dynamic programming)

Self-check

1. Why can you not binary-search a pile of unmarked exam scripts on a desk?
2. After the first pass of bubble sort on $\{8, 4, 1, 9, 3\}$, which value is guaranteed to be in its final seat?
3. Write three-line pseudocode that counts how many even numbers sit in array $A[1..n]$.

Answers. (1) They are not sorted (or not randomly accessible as an indexed array). (2) The maximum, 9, sits at the end. (3) $count \leftarrow 0$; FOR $i \leftarrow 1$ TO n DO IF $A[i]$ MOD $2 = 0$ THEN $count \leftarrow count + 1$.

Recap

1. Think: decompose → pattern → abstract → algorithm → (later) code.
2. Algorithm = language-free steps; pseudocode = structured English.
3. Bubble: neighbour swaps, stable, $O(n^2)$.
4. Selection: plant the next minimum, few swaps, $O(n^2)$.
5. Linear search: unsorted, $O(n)$. Binary search: sorted array, $O(\log n)$, $MID = \lfloor (BEG + END)/2 \rfloor$.
6. Judge every algorithm on correctness, efficiency, and clarity.